

MAE 263F – Homework 1

Yuchen Wang (Ph.D. in Mechanical Engineering)

University of California, Los Angeles

October 11, 2025

I. PSEUDOCODE AND CODE STRUCTURE

Question: Write pseudocode for your simulator describing its main logic and workflow. Briefly describe the main functions and scripts in your implementation (2–3 sentences each), including their inputs and outputs. Create a simple block diagram showing how the functions or scripts interact (i.e., which one calls which).

Answer:

Pseudocode:

- 1) **Input:** Node file (`nodes.txt`), spring file (`springs.txt`)
- 2) **Initialization:**
 - a) Read node coordinates
 - b) Read spring connectivity and compute rest lengths ℓ_k
 - c) Assemble global vectors of position $\mathbf{x}$ and velocity $\mathbf{u}$
 - d) Define mass vector $\mathbf{m}$, stiffness constants, and time step Δt
- 3) **Time Integration Loop:**
 - a) For each time step $t_n \rightarrow t_{n+1}$:
 - i) Predict $\mathbf{x}_{n+1}^{(0)} \leftarrow \mathbf{x}_n$
 - ii) Compute internal force $\mathbf{f}$ and Jacobian $\mathbf{J}$ via `getForceJacobian`
 - iii) Solve $\mathbf{J}\Delta\mathbf{x} = \mathbf{f}$ using Newton–Raphson iteration
 - iv) Update $\mathbf{x}_{n+1} \leftarrow \mathbf{x}_{n+1}^{(0)} - \Delta\mathbf{x}$
 - v) Update velocity $\mathbf{u}_{n+1} = (\mathbf{x}_{n+1} - \mathbf{x}_n)/\Delta t$
 - vi) Record y-coordinate of free nodes
 - vii) If $t_{n+1} \in \{0, 0.1, 1, 10, 100\}$, plot the current configuration
- 4) **Post-Processing:** Plot $y(t)$ histories of free nodes.

Main Functions:

- 1) **Homework1.py:** Main driver script that reads input data, initializes system parameters, runs the implicit Euler integration loop, and generates plots of nodal motion. Inputs: node and spring files. Outputs: time histories and snapshots.
- 2) **gradEs:** Computes the gradient (first derivative) of the stretching energy between two connected nodes with respect to their coordinates, giving the direction and magnitude of the elastic restoring force.
- 3) **hessEs:** Computes the Hessian matrix (second derivatives) of that same stretching energy, describing how the gradient changes.

- 4) **getForceJacobian:** assemble the total force vector and its Jacobian matrix for a dynamic mass–spring system at a given time step. It combines inertial, elastic, and external forces into one residual $\mathbf{f}$ and corresponding stiffness matrix $\mathbf{J}$ used for solving motion via Newton–Raphson iteration.
- 5) **myInt:** Performs one implicit Euler integration step using Newton–Raphson iteration, returning the updated positions $\mathbf{x}_{n+1}$ and velocities $\mathbf{u}_{n+1}$.
- 6) **plot:** Visualizes the network configuration at a given time step using nodal coordinates and spring connectivity.

Code Structure Diagram:

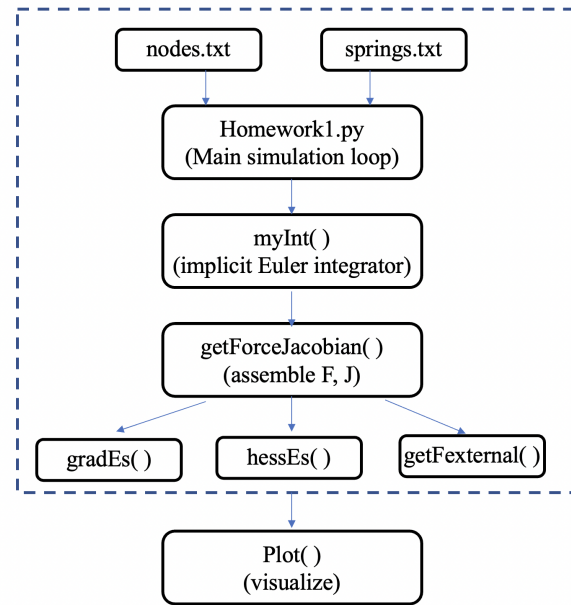


Fig. 1. Block diagram showing how the functions interaction

The simulation begins by reading nodal coordinates from `nodes.txt` and spring connectivity and stiffness from `springs.txt`. The main script (`Homework1.py`) initializes all parameters and iteratively calls the implicit Euler integrator (`myInt`) at each time step. Inside `myInt`, the global internal force and Jacobian are computed by `getForceJacobian`, which in turn calls `gradEs` and `hessEs` to evaluate the spring forces

and stiffness matrices, and `getFexternal` to include gravity. After each update, `plot` visualizes the network configuration. The overall workflow ensures a consistent coupling between geometry, physics, and visualization.

II. CHOOSE AN APPROPRIATE TIME STEP

Question: How do you choose an appropriate time step size Δt ?

Answer:

For our implicit first-order integrator, using a larger Δt speeds up computation but introduces phase lag and artificial damping, while a smaller Δt improves accuracy at the cost of longer runtime. A practical rule is to use the largest Δt that keeps errors within a reasonable tolerance.

When comparing different time step sizes:

- $\Delta t = 0.01$ s: produces smooth and accurate oscillations that decay naturally, closely matching the expected physical damping, but it takes about 16 s to finish. (Shown in Fig. 2.)

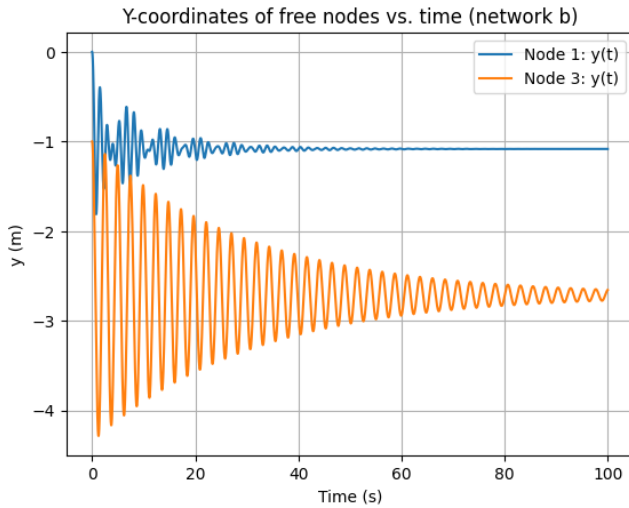


Fig. 2. Y-coordinates of all free nodes vs. time for $\Delta t = 0.01$ s

- $\Delta t = 0.1$ s: still captures the oscillations and damping trend well, taking only 1 s, so it offers a good balance between accuracy and efficiency. (Shown in Fig. 3.)
- $\Delta t = 1$ s: runs fastest but quickly forces the system to its equilibrium without visible oscillations, showing significant numerical damping and loss of accuracy. (Shown in Fig. 4.)

Hence, $\Delta t = 0.1$ s is the most appropriate choice—it is much faster than 0.01 s while maintaining physically reasonable accuracy.

III. EXPLICIT EULER SIMULATION

Question: Simulate the spring network using *Explicit Euler* for $t \in [0, 1]$ s. If it becomes unstable, reduce Δt and try again. If it still fails even at $\Delta t = 10^{-6}$ s, state this in the report. Explain which method (implicit vs. explicit) is preferable for this spring network and why.

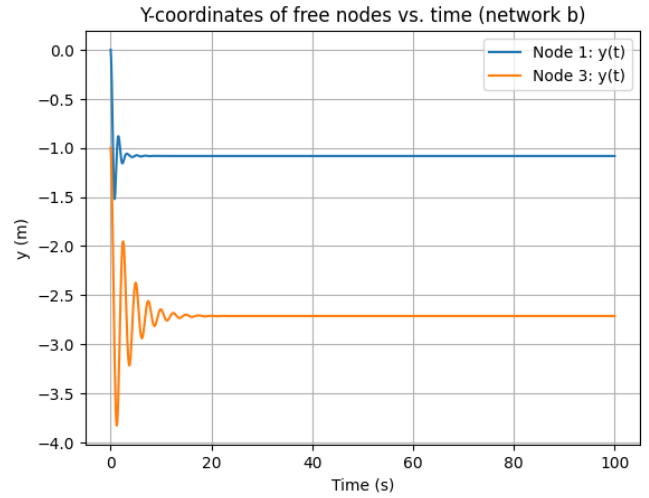


Fig. 3. Y-coordinates of all free nodes vs. time for $\Delta t = 0.1$ s

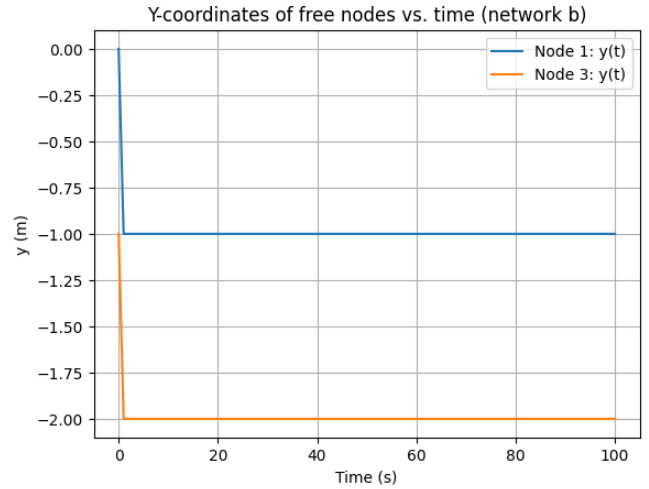


Fig. 4. Y-coordinates of all free nodes vs. time for $\Delta t = 1$ s

Answer: The explicit Euler method directly updates displacement and velocity based on the current state, but it becomes unstable for stiff systems unless the time step Δt is extremely small. In this spring network, instability occurs due to large stiffness-to-mass ratios, making implicit integration more suitable. The implicit Euler method is unconditionally stable and can handle stiff systems efficiently, although it may introduce numerical damping.

I added a new cell implementing the explicit Euler method to the implicit code. I tested several time steps ($\Delta t = 1, 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}$), but all of them led to instability (the result for $\Delta t = 10^{-6}$ is shown in Fig. 5, and it took more than 60 s to compute). I think the reason why the y-displacement does not match the implicit result is that the explicit method cannot properly capture the equilibrium configuration for such a stiff system — the time step is still too large relative to the system's natural frequency, causing accumulated numerical errors and divergence instead of converging to a stable deformation.

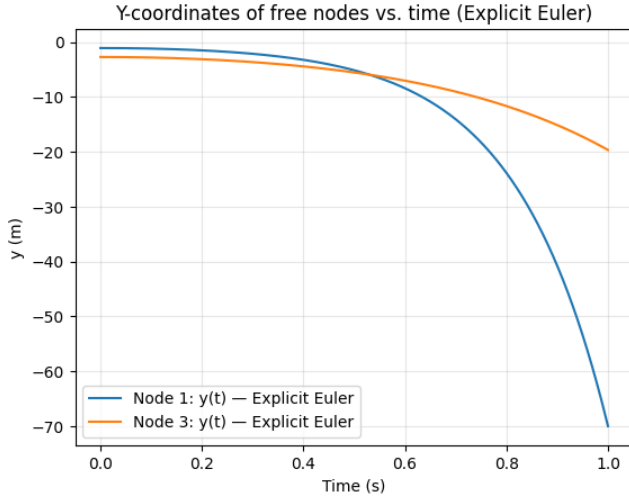


Fig. 5. Y-coordinates of all free nodes from the explicit Euler method vs. time for $\Delta t = 10^{-6}$ s

IV. NUMERICAL DAMPING IN IMPLICIT EULER

Question: The simulation with implicit Euler appears to reach a static state (numerical damping). Read about the Newmark- β family of time integrators and explain how such integrators can mitigate such artificial damping.

Answer: The Newmark- β family of time integrators is a widely used class of algorithms for solving second-order equations of motion in structural dynamics:

$$M\ddot{u} + C\dot{u} + Ku = F(t)$$

It defines displacement and velocity at the next time step using two parameters, β and γ :

$$u_{n+1} = u_n + \Delta t \dot{u}_n + \Delta t^2 \left[\left(\frac{1}{2} - \beta \right) \ddot{u}_n + \beta \ddot{u}_{n+1} \right],$$

$$\dot{u}_{n+1} = \dot{u}_n + \Delta t \left[(1 - \gamma) \ddot{u}_n + \gamma \ddot{u}_{n+1} \right].$$

Different combinations of (β, γ) correspond to various integration schemes:

- $\beta = 0, \gamma = 1/2$: central difference (explicit, no numerical damping),
- $\beta = 1/6, \gamma = 1/2$: linear acceleration (conditionally stable),
- $\beta = 1/4, \gamma = 1/2$: constant average acceleration (implicit, unconditionally stable),
- $\beta > 1/4$: introduces numerical damping to suppress high-frequency oscillations.

In contrast to the implicit Euler method, which introduces excessive numerical damping and drives the solution quickly to a static state, the Newmark- β method allows the user to control numerical damping through the parameters β and γ . By choosing appropriate values (e.g., $\beta = 1/4, \gamma = 1/2$), one can achieve unconditional stability while maintaining accurate low-frequency dynamics.

V. NEWMARK- β SIMULATION

Question: Simulate the spring networks in (a) and (b) using the Newmark- β method and demonstrate that the artificial damping observed in the implicit Euler method is eliminated.

Answer:

I simulate the spring networks (a) and (b) with the Newmark- β time integrator for the second-order system, using the average-acceleration parameters $\beta = \frac{1}{4}$ and $\gamma = \frac{1}{2}$, which are unconditionally stable and introduce no algorithmic damping for linear dynamics.

I use $\Delta t = 0.1$ s and report responses up to $t = 10$ s and $t = 100$ s for spring net (a) and spring net (b), respectively. In both networks the Newmark- β solutions exhibit sustained, physically consistent oscillations about the equilibrium without spurious decay, thereby *eliminating the artificial damping* observed with implicit Euler (which over-dissipates energy and drives the response to a static state).

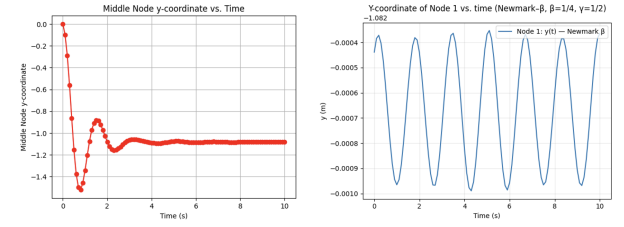


Fig. 6. Comparison of results between the implicit Euler method and the Newmark- β method for spring network (a)

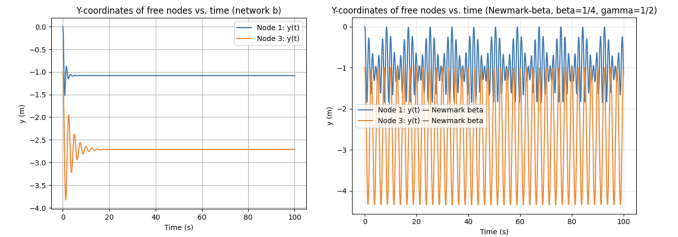


Fig. 7. Comparison of results between the implicit Euler method and the Newmark- β method for spring network (b)

CODE AND SOURCE ATTRIBUTION

The source code is adapted from *Lecture 3 – Spring Network Simulation* and has been refined and formatted with the assistance of ChatGPT.